



INSTITUTO DE  
INVESTIGACIONES  
EN MATEMÁTICAS  
APLICADAS Y  
EN SISTEMAS

Plataforma  
NoSQL para  
Inteligencia de  
Negocio  
Data Seekers



Universidad Nacional Autónoma de México

Instituto de Investigaciones en Ma-  
temáticas Aplicadas y en Sistemas

## Bases de Datos No Estructuradas

**Integrantes del Equipo**

Aldo Sebastián Altamirano Vázquez

Carlo Kiliano Ferrera Guadarrama

José Julian Pérez Bustamante

Isaac Pancardo Botello

Leonardo Negrete Mancera

**Semestre: 2026-2**

## Resumen

El presente reporte documenta el diseño, implementación y evaluación de *Panam*, una plataforma de inteligencia de negocio orientada al sector de retail de calzado en México. El proyecto fue desarrollado por el equipo Data Seekers como producto final de la asignatura de Bases de Datos NoSQL, simulando un escenario consultivo en el que una marca real de calzado mexicano requería transformar su capacidad analítica mediante tecnologías no relacionales. La solución propuesta integra una arquitectura híbrida que combina *MongoDB* —para el almacenamiento de datos semi-estructurados como reseñas de usuarios y eventos web— con *Apache Cassandra* —optimizada para series temporales de ventas e inventario—, articuladas mediante una capa de servicios construida en *FastAPI*. El pipeline de datos comprende cuatro etapas: generación vectorizada de un dataset sintético de 1.75 millones de registros, extracción-transformación-carga (ETL) con análisis de sentimiento en español mediante un lexicón propio, persistencia poliglota en ambos motores NoSQL, y visualización ejecutiva a través de dos dashboards (Tableau Public y un *frontend* HTML modular desplegado en GitHub Pages). Los resultados demuestran que la arquitectura propuesta logra tiempos de respuesta inferiores a 50 ms en consultas analíticas críticas y permite escalar de manera horizontal sin rediseño del esquema. Se concluye que la adopción de bases de datos NoSQL, lejos de ser una decisión meramente técnica, constituye una palanca estratégica para empresas de retail que buscan operar bajo paradigmas *data-driven*.

**Palabras clave:** MongoDB, Cassandra, NoSQL, ETL, dashboard, análisis de sentimiento, retail, Panam, FastAPI, persistencia poliglota.

## Índice

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                | <b>7</b>  |
| 1.1. Contexto académico . . . . .                                     | 7         |
| 1.2. Estructura del reporte . . . . .                                 | 8         |
| <b>2. Problemática y objetivos</b>                                    | <b>8</b>  |
| 2.1. Planteamiento del problema . . . . .                             | 8         |
| 2.2. Objetivo general . . . . .                                       | 9         |
| 2.3. Objetivos específicos . . . . .                                  | 9         |
| 2.4. Alcance y delimitaciones . . . . .                               | 10        |
| <b>3. Marco teórico</b>                                               | <b>10</b> |
| 3.1. Bases de datos NoSQL: orígenes y taxonomía . . . . .             | 10        |
| 3.2. El teorema CAP y sus implicaciones de diseño . . . . .           | 11        |
| 3.3. Persistencia poliglota . . . . .                                 | 11        |
| 3.4. Análisis de sentimiento sobre texto en español . . . . .         | 11        |
| 3.5. Visualización analítica y <i>dashboards</i> ejecutivos . . . . . | 12        |
| <b>4. Metodología</b>                                                 | <b>13</b> |
| 4.1. Enfoque general . . . . .                                        | 13        |
| 4.2. Fases del proyecto . . . . .                                     | 13        |
| 4.3. Herramientas de colaboración . . . . .                           | 14        |
| 4.4. Stack tecnológico . . . . .                                      | 14        |
| 4.5. Gestión del repositorio . . . . .                                | 15        |
| <b>5. Arquitectura del sistema</b>                                    | <b>16</b> |
| 5.1. Visión general . . . . .                                         | 16        |
| 5.2. Separación de responsabilidades entre motores . . . . .          | 16        |
| 5.3. Modelo conceptual de datos . . . . .                             | 17        |
| 5.4. Patrones de acceso anticipados . . . . .                         | 17        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>6. Generación de datos sintéticos</b>                          | <b>18</b> |
| 6.1. Motivación y contexto . . . . .                              | 18        |
| 6.2. Trabajo previo del equipo . . . . .                          | 18        |
| 6.3. El script <code>generar_datos_masivos.py</code> . . . . .    | 19        |
| 6.4. Volúmenes generados . . . . .                                | 20        |
| 6.5. Distribuciones estadísticas implementadas . . . . .          | 21        |
| 6.5.1. Estacionalidad de ventas . . . . .                         | 21        |
| 6.5.2. Dominancia regional . . . . .                              | 21        |
| 6.5.3. Ranking de modelos . . . . .                               | 22        |
| 6.5.4. Calificaciones y sentimiento . . . . .                     | 22        |
| 6.6. Cumplimiento del esquema esperado por el ETL . . . . .       | 22        |
| 6.7. Coexistencia de las dos rutas de generación . . . . .        | 23        |
| 6.8. Pipeline completo del proyecto . . . . .                     | 23        |
| 6.9. Reproducibilidad . . . . .                                   | 25        |
| <b>7. ETL: transformaciones y análisis de sentimiento</b>         | <b>25</b> |
| 7.1. Visión general del <i>pipeline</i> ETL . . . . .             | 25        |
| 7.2. Lexicón propio en español . . . . .                          | 25        |
| 7.3. Algoritmo de clasificación . . . . .                         | 26        |
| 7.4. Campos derivados . . . . .                                   | 26        |
| 7.5. Predicción de <i>stockout</i> . . . . .                      | 27        |
| 7.6. Justificación metodológica del enfoque por lexicón . . . . . | 27        |
| <b>8. Modelado NoSQL e implementación de la persistencia</b>      | <b>27</b> |
| 8.1. Levantamiento de servicios con Docker Compose . . . . .      | 27        |
| 8.2. Modelado en MongoDB . . . . .                                | 28        |
| 8.2.1. Colección <code>resenas_enriquecidas</code> . . . . .      | 28        |
| 8.2.2. Colección <code>eventos_web</code> . . . . .               | 29        |
| 8.3. Modelado en Cassandra . . . . .                              | 30        |
| 8.3.1. Tabla <code>ventas_por_fecha</code> . . . . .              | 30        |
| 8.3.2. Tabla <code>inventario_por_sucursal</code> . . . . .       | 30        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 8.4. Consistencia y replicación . . . . .                           | 30        |
| 8.5. Scripts de carga . . . . .                                     | 30        |
| <b>9. Dashboards de visualización</b>                               | <b>31</b> |
| 9.1. Estrategia de doble entrega . . . . .                          | 31        |
| 9.2. Dashboard en Tableau Public . . . . .                          | 31        |
| 9.2.1. Composición y elementos . . . . .                            | 31        |
| 9.2.2. Diseño visual . . . . .                                      | 32        |
| 9.2.3. Limitaciones de Tableau Public . . . . .                     | 33        |
| 9.3. Dashboard HTML modular . . . . .                               | 33        |
| 9.3.1. Decisiones tecnológicas . . . . .                            | 33        |
| 9.3.2. Arquitectura modular . . . . .                               | 34        |
| 9.3.3. Pipeline de datos . . . . .                                  | 35        |
| 9.3.4. Cross-filtering . . . . .                                    | 36        |
| 9.3.5. Estética visual . . . . .                                    | 37        |
| 9.3.6. Iteraciones del diseño . . . . .                             | 38        |
| 9.3.7. Mapa coroplético: implementación y discontinuación . . . . . | 38        |
| 9.4. Despliegue en GitHub Pages . . . . .                           | 39        |
| 9.5. Cumplimiento de la rúbrica . . . . .                           | 40        |
| <b>10.Resultados y evaluación</b>                                   | <b>41</b> |
| 10.1. Volumen procesado . . . . .                                   | 41        |
| 10.2. Tiempos de procesamiento . . . . .                            | 41        |
| 10.3. Métricas del dashboard . . . . .                              | 42        |
| 10.4. Insights observados . . . . .                                 | 42        |
| 10.5. Cumplimiento de la rúbrica . . . . .                          | 43        |
| 10.6. Limitaciones reconocidas . . . . .                            | 44        |
| <b>11.Conclusiones y trabajo futuro</b>                             | <b>45</b> |
| 11.1. Conclusiones . . . . .                                        | 45        |
| 11.2. Trabajo futuro . . . . .                                      | 45        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| 11.3. Reflexión final . . . . .                                         | 46        |
| <b>A. Anexos</b>                                                        | <b>47</b> |
| A.1. Anexo A: Estructura completa del repositorio . . . . .             | 47        |
| A.2. Anexo B: Comandos de levantamiento del entorno . . . . .           | 49        |
| A.3. Anexo C: Variables del archivo <code>.env.example</code> . . . . . | 50        |
| A.4. Anexo D: Información del equipo . . . . .                          | 50        |
| A.5. Anexo E: Enlaces relevantes . . . . .                              | 51        |

## Índice de tablas

|     |                                                                   |    |
|-----|-------------------------------------------------------------------|----|
| 1.  | Pila tecnológica del proyecto Panam. . . . .                      | 15 |
| 2.  | Reparto de entidades del dominio entre los motores NoSQL. . . . . | 17 |
| 3.  | Volúmenes de datos generados en escala <b>full</b> . . . . .      | 20 |
| 4.  | Estructura de la colección <b>resenas_enriquecidas</b> . . . . .  | 29 |
| 5.  | Módulos JavaScript del dashboard HTML. . . . .                    | 35 |
| 6.  | Paleta cromática del dashboard HTML. . . . .                      | 37 |
| 7.  | Cumplimiento de los requerimientos de la rúbrica. . . . .         | 41 |
| 8.  | Tiempos de ejecución por etapa del <i>pipeline</i> . . . . .      | 42 |
| 9.  | Cumplimiento de la rúbrica de evaluación. . . . .                 | 44 |
| 10. | Integrantes del equipo Data Seekers. . . . .                      | 50 |

## Índice de figuras

|    |                                                                                                                            |    |
|----|----------------------------------------------------------------------------------------------------------------------------|----|
| 1. | Diagrama simplificado de la arquitectura Panam. . . . .                                                                    | 16 |
| 2. | Salida del script <b>generar_datos_masivos.py</b> en escala <b>full</b> . . . . .                                          | 21 |
| 3. | <i>Pipeline</i> completo de datos del proyecto. . . . .                                                                    | 24 |
| 4. | Dashboard final en Tableau Public con paleta oscura y KPI en colores plenos. . . . .                                       | 32 |
| 5. | Ejemplo de <i>cross-filtering</i> activo: clic en una barra del ranking propaga el filtro a todos los componentes. . . . . | 36 |
| 6. | Vista superior del dashboard HTML: <i>hero</i> , filtros, KPI y gráfico de tendencia. . . . .                              | 38 |
| 7. | Dashboard HTML hospedado en GitHub Pages. . . . .                                                                          | 40 |

## 1. Introducción

El sector de retail de calzado en México opera bajo condiciones de alta variabilidad de la demanda, fuerte presión competitiva y un creciente volumen de información generada en canales digitales. En este contexto, las marcas que aspiran a sostener márgenes operativos y construir lealtad de marca han migrado progresivamente desde modelos de gestión basados en intuición hacia arquitecturas analíticas que permiten la toma de decisiones soportada en datos. Sin embargo, la heterogeneidad de las fuentes —transacciones puntuales, eventos web, reseñas de usuarios en redes sociales, registros de inventario— vuelve insuficiente al modelo relacional clásico para responder con la agilidad y la escalabilidad que el negocio requiere (Harrison, 2015).

El presente trabajo aborda este desafío mediante un caso de estudio aplicado a *Panam*, marca emblemática de calzado mexicano. El equipo Data Seekers diseñó e implementó una plataforma de inteligencia de negocio que combina dos motores NoSQL —*MongoDB* y *Apache Cassandra*— bajo el paradigma de persistencia poliglota (Sadalage & Fowler, 2012). La motivación de esta elección es explícita: cada base de datos resuelve un subproblema distinto del dominio (almacenamiento documental flexible vs. series temporales de alta cardinalidad) y la combinación de ambas supera lo que cualquiera podría ofrecer de manera aislada.

### 1.1 Contexto académico

El reporte se inscribe en el marco de la asignatura de Bases de Datos NoSQL impartida por la profesora María Lourdes Soto Cortés. La rúbrica de evaluación contempla cinco componentes principales: (a) una presentación ejecutiva con narrativa de caso de éxito; (b) la demostración en vivo de los motores NoSQL operando; (c) la ejecución del proceso ETL frente a la audiencia; (d) la exhibición de índices y agregados; y (e) la entrega de un *dashboard* con al menos cinco gráficas, cuatro *Key Performance Indicators* (KPI), una tabla resumen y tres variables o filtros sincronizados. El presente documento técnico complementa estas entregas con el sustento conceptual, metodológico y empírico que permite trazar las decisiones de diseño tomadas a lo largo del proyecto.



## 1.2 Estructura del reporte

El documento se organiza en trece secciones temáticas. La sección 2 formaliza el problema de negocio y enuncia los objetivos del proyecto. La sección 3 desarrolla el marco teórico que sustenta las decisiones tecnológicas. La sección 4 describe la metodología de trabajo del equipo. Las secciones 5 a 9 presentan los componentes técnicos del sistema en orden de *pipeline*: arquitectura general, generación de datos, ETL, modelado NoSQL, capa de servicios y visualización. La sección 10 consolida los resultados experimentales y la sección 11 cierra con conclusiones y trabajo futuro.

## 2. Problemática y objetivos

### 2.1 Planteamiento del problema

Los retailers de calzado en mercados emergentes enfrentan simultáneamente tres tensiones operativas que erosionan la rentabilidad y la experiencia del cliente. En primer lugar, la *imprevisibilidad del desabastecimiento* genera pérdidas directas de venta y costos indirectos derivados de la frustración del consumidor, quien con frecuencia migra a competidores ante una experiencia fallida de búsqueda de stock. En segundo lugar, el *desconocimiento de la reputación digital* en plataformas como Instagram, TikTok y Twitter impide diseñar campañas de mercadotecnia que respondan al sentir real de la base de clientes, lo cual deriva en inversiones publicitarias con tasas de conversión subóptimas. En tercer lugar, la *toma de decisiones basada en intuición*, sostenida en reportes mensuales o trimestrales, contrasta con la velocidad a la que se modifican las preferencias del consumidor contemporáneo, particularmente en segmentos juveniles donde las tendencias tienen ciclos de vida de pocas semanas.

Estas tensiones convergen en un problema técnico concreto: la incapacidad de las arquitecturas relacionales tradicionales para procesar, almacenar y servir, con baja latencia, datos heterogéneos provenientes de fuentes diversas. Las reseñas de usuarios contienen texto libre, emojis y *hashtags*; los registros de ventas son intrínsecamente temporales y de alta cardinalidad; los eventos web exhiben esquemas variables; y los datos de inventario requieren agregaciones rápidas por modelo, talla y sucursal. Forzar este conjunto a un único modelo tabular implica concesiones de rendimiento y de flexibilidad que el negocio no puede permitirse en un horizonte de crecimiento.

## 2.2 Objetivo general

Diseñar, implementar y evaluar una plataforma de inteligencia de negocio basada en bases de datos NoSQL que permita a una marca de retail de calzado integrar fuentes de datos heterogéneas, procesarlas mediante un *pipeline* ETL automatizado y entregar valor analítico a usuarios ejecutivos a través de un *dashboard* interactivo, todo ello bajo principios de escalabilidad horizontal, persistencia poliglota y baja latencia de consulta.

## 2.3 Objetivos específicos

1. **Generar un *dataset* sintético representativo** del dominio de retail de calzado mexicano, con volumen suficiente para evidenciar las ventajas de las arquitecturas NoSQL frente a alternativas relacionales (objetivo cuantitativo: superar el millón de registros agregados entre las cuatro entidades del modelo).
2. **Implementar un *pipeline* ETL** que enriquezca los datos crudos con análisis de sentimiento aplicado a texto en español y con predicciones tempranas de *stockout*, sin dependencia de servicios externos.
3. **Modelar el dominio bajo principios de persistencia poliglota**, asignando cada tipo de dato al motor NoSQL más adecuado (MongoDB para reseñas y eventos; Cassandra para ventas e inventario).
4. **Construir una capa de servicios REST** mediante FastAPI que exponga, con documentación automática, los puntos de acceso necesarios para alimentar la capa de visualización.
5. **Entregar dos dashboards complementarios**: uno en Tableau Public y otro en HTML/CS-S/JavaScript desplegado en GitHub Pages, ambos cumpliendo con el mínimo de cinco gráficas, cuatro KPI, una tabla resumen y tres filtros sincronizados.
6. **Documentar el proyecto de manera reproducible**, de modo que un evaluador externo pueda levantar la plataforma localmente mediante **docker-compose** y reproducir las consultas en vivo sin asistencia.

## 2.4 Alcance y delimitaciones

El proyecto se circunscribe a un escenario de *prueba de concepto* (PoC) ejecutado sobre datos sintéticos generados de manera controlada. No contempla la integración con sistemas transaccionales reales de la marca, ni la implementación de mecanismos de autenticación de grado producción, ni la operación bajo cargas distribuidas reales en múltiples nodos. La elección de Tableau Public como una de las salidas de visualización conlleva una limitación deliberada respecto a la confidencialidad de los datos, mitigada por la naturaleza sintética del *dataset*.

## 3. Marco teórico

### 3.1 Bases de datos NoSQL: orígenes y taxonomía

El término NoSQL —acrónimo de *Not Only SQL*— designa a un conjunto de tecnologías de gestión de datos surgidas a partir de la primera década del siglo XXI como respuesta a las limitaciones de los sistemas relacionales frente a cargas web de gran escala (Harrison, 2015). La motivación inicial fue empírica: empresas como Google, Amazon y Facebook constataron que, al alcanzar volúmenes del orden de petabytes y latencias submilisegundo en operaciones globales, el modelo relacional —con su énfasis en la consistencia transaccional y los esquemas rígidos— imponía costos de rendimiento y operación incompatibles con sus modelos de negocio.

La literatura especializada (Harrison, 2015; Sadalage & Fowler, 2012) clasifica los sistemas NoSQL en cuatro grandes familias, cada una con un caso de uso paradigmático:

1. **Documentales**, que almacenan colecciones de documentos semi-estructurados (típicamente en formato JSON o BSON). Ejemplos representativos incluyen MongoDB, CouchDB y Amazon DocumentDB.
2. **De columnas anchas** (*wide-column*), optimizadas para series temporales y datos con esquemas de alta cardinalidad. Apache Cassandra y HBase son sus exponentes más destacados.
3. **De pares clave-valor**, orientadas a operaciones de búsqueda directa con latencias mínimas. Redis y DynamoDB lideran esta categoría.

4. **De grafos**, especializadas en relaciones complejas entre entidades. Neo4j y Amazon Neptune son referentes de esta familia.

### 3.2 El teorema CAP y sus implicaciones de diseño

La selección entre motores NoSQL no puede comprenderse cabalmente sin recurrir al teorema CAP formulado por Brewer (Brewer, 2000). Este teorema establece que, en un sistema distribuido, resulta imposible garantizar simultáneamente las tres propiedades de Consistencia (*Consistency*), Disponibilidad (*Availability*) y Tolerancia a particiones (*Partition tolerance*); cualquier diseño debe sacrificar, al menos parcialmente, una de las tres.

MongoDB se posiciona tradicionalmente como un sistema CP: privilegia la consistencia y la tolerancia a particiones, lo que lo hace adecuado para escenarios donde la integridad de cada documento es crítica. Cassandra, por su parte, adopta un perfil AP: prioriza la disponibilidad sobre la consistencia estricta, ofreciendo modelos de *consistencia eventual* que escalan horizontalmente con facilidad y soportan operación multi-región sin un nodo coordinador único. Esta diferencia, lejos de ser trivial, justifica la elección de una arquitectura híbrida en el presente proyecto: las reseñas y eventos toleran consistencia eventual con mayor facilidad que un sistema de inventario donde la sobreventa puede tener consecuencias financieras directas.

### 3.3 Persistencia poliglota

El concepto de *persistencia poliglota* (*polyglot persistence*), introducido formalmente por Fowler y Sadalage (Sadalage & Fowler, 2012), plantea que las aplicaciones modernas raramente se ven bien servidas por una única tecnología de almacenamiento. En lugar de ello, los sistemas robustos suelen combinar varias bases de datos especializadas, cada una asignada al subproblema que mejor resuelve. Este enfoque acepta como contraparte una mayor complejidad operativa —se requieren múltiples motores, múltiples *drivers*, múltiples copias de seguridad— a cambio de rendimientos y flexibilidades sustancialmente superiores en cada uno de los subsistemas.

### 3.4 Análisis de sentimiento sobre texto en español

El análisis de sentimiento, también denominado *opinion mining*, constituye una subdisciplina del Procesamiento de Lenguaje Natural (PLN) cuyo objetivo es clasificar fragmentos de texto en

categorías afectivas —positivo, negativo, neutro— y, en versiones más avanzadas, en gradientes continuos de polaridad (Pang & Lee, 2008). Las aproximaciones técnicas se agrupan en tres grandes familias: (a) métodos basados en lexicón, que asignan puntuaciones predefinidas a palabras y emojis; (b) métodos de aprendizaje supervisado clásico, que entrenan clasificadores sobre corpus etiquetados; y (c) métodos basados en aprendizaje profundo, particularmente arquitecturas *transformer* como BERT, que han redefinido el estado del arte en años recientes.

En el contexto del presente proyecto se optó por un enfoque basado en lexicón, por dos razones principales. Primero, la disponibilidad de modelos pre-entrenados en español de calidad equivalente al inglés era, al momento del desarrollo, sustancialmente menor; herramientas comúnmente recomendadas como *TextBlob* presentan un soporte deficiente para nuestro idioma. Segundo, un lexicón curado para el dominio específico de calzado y *retail* ofrece interpretabilidad —cada decisión del clasificador puede trazarse a una palabra concreta—, virtud que un modelo de aprendizaje profundo difícilmente igualaría sin esfuerzos sustanciales de explicabilidad.

### 3.5 Visualización analítica y *dashboards* ejecutivos

Un *dashboard* ejecutivo es, en términos de Few (Few, 2006), “una pantalla visual de la información más importante necesaria para alcanzar uno o más objetivos, consolidada y organizada en una sola pantalla de modo que la información puede ser monitoreada de un vistazo”. Esta definición, aparentemente sencilla, encierra principios de diseño rigurosos: la información debe ser jerarquizada, los KPI deben elegirse en función de su capacidad de orientar la acción, y los gráficos deben minimizar la *razón tinta/datos* (*data-ink ratio*) en el sentido de Tufte (Tufte, 2001).

En el ámbito del retail, los *dashboards* ejecutivos suelen incluir métricas como ingresos totales, ticket promedio, tasa de conversión, unidades vendidas, sentimiento agregado de la base de clientes y alertas tempranas de inventario crítico. La presente plataforma adopta estas convenciones e integra adicionalmente capacidades de *cross-filtering*: al hacer clic en cualquier componente del *dashboard*, el resto se actualiza para reflejar el subconjunto seleccionado.

## 4. Metodología

### 4.1 Enfoque general

El proyecto se desarrolló bajo una metodología incremental e iterativa, con sprints semanales coordinados a través del repositorio en GitHub. El equipo Data Seekers adoptó un *flujo* de trabajo basado en ramas (*branch-based workflow*): cada miembro trabajaba sobre una rama temática (por ejemplo, `feat/datos-masivos-dashboard` o `feat/dashboard-html-modular`) y, al concluir su tarea, abría un *pull request* (PR) hacia `main` que era revisado por al menos un compañero antes de su integración. Esta práctica, característica de los equipos de desarrollo profesional, permitió mantener trazabilidad de cada cambio y facilitar la corrección temprana de errores.

### 4.2 Fases del proyecto

El trabajo se estructuró en seis fases secuenciales, aunque con solapamientos deliberados que permitieron paralelizar tareas:

1. **Fase de descubrimiento.** Análisis del dominio (retail de calzado mexicano), revisión de la rúbrica de evaluación y selección preliminar de tecnologías.
2. **Fase de diseño.** Definición de la arquitectura, modelado conceptual de las cuatro entidades del dominio (ventas, reseñas, inventario, eventos) y selección de motores NoSQL.
3. **Fase de generación de datos.** Construcción de un script vectorizado en *NumPy* capaz de producir más de un millón de registros en segundos, con tres escalas configurables (`demo`, `media`, `full`).
4. **Fase de ETL.** Implementación del *pipeline* de transformación, incluyendo análisis de sentimiento por lexicón y predicción de *stockout*.
5. **Fase de persistencia.** Carga de los datos transformados a MongoDB y Cassandra mediante scripts dedicados, con configuración por `docker-compose`.
6. **Fase de visualización y entrega.** Construcción de dos dashboards (Tableau Public y HTML modular), documentación técnica y preparación de la presentación ejecutiva.

### 4.3 Herramientas de colaboración

- **Repositorio.** GitHub, ubicado en <https://github.com/CarloKiliano/data-seekers>.
- **Despliegue del dashboard.** GitHub Pages, con publicación automática desde la carpeta `/docs` de la rama `main`.
- **Comunicación.** Mensajería instantánea sincrónica para coordinación cotidiana y discusiones técnicas en los hilos de *pull request*.
- **Entornos de desarrollo.** Visual Studio Code y JupyterLab para el desarrollo de notebooks de demostración.

### 4.4 Stack tecnológico

La pila tecnológica adoptada se resume en la Tabla 1. La elección de cada componente respondió a criterios de adecuación al problema, madurez del ecosistema, disponibilidad de documentación y compatibilidad con *macOS* sobre arquitectura Apple Silicon (M1/M2), entorno predominante en el equipo.

**Tabla 1.** Pila tecnológica del proyecto Panam.

| Capa                | Tecnología               | Justificación                                    |
|---------------------|--------------------------|--------------------------------------------------|
| Base documental     | MongoDB 6.0              | Almacenamiento flexible de reseñas y eventos.    |
| Base columnar       | Apache Cassandra 4.0     | Series temporales de ventas e inventario.        |
| Orquestación        | Docker Compose           | Levantamiento reproducible de servicios.         |
| Generación de datos | Python 3.11 + NumPy      | Vectorización para alto rendimiento.             |
| ETL                 | Pandas + lexicón propio  | Procesamiento tabular y análisis de sentimiento. |
| API                 | FastAPI + Uvicorn        | Documentación automática y rendimiento.          |
| Notebooks           | JupyterLab               | Demostración en vivo de operaciones.             |
| Dashboard 1         | Tableau Public           | Estándar visual ejecutivo.                       |
| Dashboard 2         | HTML + CSS + Chart.js v4 | Hosting libre en GitHub Pages.                   |
| Despliegue web      | GitHub Pages             | Costo cero, integración con repositorio.         |

#### 4.5 Gestión del repositorio

La estructura del repositorio refleja una separación clara de responsabilidades, como se ilustra en la Figura ?? y se detalla en los anexos. Los directorios principales son:

- `api/`: implementación del servicio FastAPI y sus enrutadores.
- `dashboards/`: archivos Tableau (`.twbx`) y dashboard Streamlit auxiliar.
- `data/`: archivos CSV generados (ignorados por Git cuando superan los 100 MB).
- `docs/`: dashboard HTML modular para GitHub Pages.
- `notebooks/`: cinco notebooks Jupyter para demostración en vivo.
- `scripts/`: tres scripts Python (generación masiva y cargas a MongoDB y Cassandra).
- `presentacion/`: materiales de la presentación final.



## 5. Arquitectura del sistema

### 5.1 Visión general

La plataforma Panam se concibe como un sistema en capas que materializa el principio de *persistencia poliglota* discutido en el marco teórico. La Figura 1 —reproducible textualmente desde el archivo `README.md` del repositorio— muestra la organización de los componentes y el flujo de datos entre ellos.

Datos Crudos (CSV) -> ETL -> MongoDB / Cassandra -> API REST -> Dashboard

**Figura 1.** Diagrama simplificado de la arquitectura Panam.

El *pipeline* se compone de seis capas funcionales, cada una con responsabilidades acotadas:

1. **Capa de generación.** Produce datos sintéticos representativos del dominio en archivos CSV.
2. **Capa de extracción y transformación.** Lee los CSV crudos, enriquece reseñas con análisis de sentimiento y calcula predicciones de *stockout*.
3. **Capa de carga.** Persiste los datos enriquecidos en MongoDB y los datos transaccionales en Cassandra.
4. **Capa de servicios.** Expone los datos a través de un API REST construido con FastAPI.
5. **Capa de visualización.** Consume el API o los datos en formato JSON para alimentar dos dashboards independientes.
6. **Capa de orquestación.** Coordina el levantamiento de los servicios mediante `docker-compose`.

### 5.2 Separación de responsabilidades entre motores

La decisión de mantener dos motores NoSQL en paralelo, en lugar de migrar todo el dominio a uno solo, fue uno de los puntos de mayor discusión metodológica del equipo. La Tabla 2 sintetiza el reparto final, junto con la justificación técnica de cada asignación.

**Tabla 2.** Reparto de entidades del dominio entre los motores NoSQL.

| Entidad              | Motor     | Justificación                                                                       |
|----------------------|-----------|-------------------------------------------------------------------------------------|
| Reseñas enriquecidas | MongoDB   | Esquema semi-estructurado, campos opcionales (emojis, hashtags), análisis de texto. |
| Eventos web          | MongoDB   | Variabilidad en propiedades por tipo de evento; consultas ad hoc.                   |
| Ventas               | Cassandra | Serie temporal de alta cardinalidad; agregaciones por modelo y región.              |
| Inventario           | Cassandra | Consultas por clave compuesta (modelo, talla, sucursal).                            |

### 5.3 Modelo conceptual de datos

Las cuatro entidades del dominio se vinculan mediante atributos no normalizados —en el sentido relacional clásico— que actúan como identificadores naturales: el campo `modelo` aparece en las cuatro entidades, mientras que `estado_origen` y `fecha` habilitan análisis cruzados entre ventas y eventos. La ausencia deliberada de *foreign keys* corresponde al principio NoSQL de *desnormalización por desempeño*: en lugar de hacer *joins* costosos en tiempo de consulta, los datos se preparan en tiempo de escritura para responder a los patrones de acceso conocidos.

### 5.4 Patrones de acceso anticipados

El diseño se guió por la enumeración exhaustiva de los patrones de acceso esperados, una práctica recomendada en la modelación NoSQL (Carpenter & Hewitt, 2020). Los patrones contemplados son:

- Consulta de ventas por modelo y rango de fechas.
- Consulta de ventas por estado y rango de fechas.
- Top-N modelos por ingresos.
- Top-N modelos por unidades.
- Sentimiento promedio por modelo.

- Distribución de calificaciones por modelo.
- Identificación de reseñas de influencers (con más de 5 000 seguidores).
- Alertas de inventario por modelo y talla.
- Tendencia mensual de ingresos.
- Eventos web por tipo y dispositivo.

Cada patrón se mapea a una colección en MongoDB o a una tabla en Cassandra, con los índices o claves de partición correspondientes. El detalle se desarrolla en la sección 8.

## 6. Generación de datos sintéticos

### 6.1 Motivación y contexto

El proyecto requería un volumen de datos suficiente para evidenciar las ventajas de escalabilidad de las arquitecturas NoSQL frente a esquemas relacionales tradicionales. Una primera iteración de generación de datos, encapsulada en el *notebook* `01_generacion_datos.ipynb` y desarrollada en la fase inicial del proyecto, produjo aproximadamente 8 800 registros mediante iteración en *Python* puro. Este volumen, adecuado para demostraciones didácticas en vivo, resultaba insuficiente para sostener la narrativa de *big data analytics* que el caso de éxito requería: con menos de diez mil ventas, las visualizaciones agregadas exhibían fluctuaciones erráticas y los índices de las bases NoSQL no tenían oportunidad de evidenciar ventajas medibles. Surgió entonces la necesidad de un segundo artefacto, vectorizado y escalable, capaz de producir órdenes de magnitud superiores en tiempos competitivos.

### 6.2 Trabajo previo del equipo

Antes del desarrollo del script masivo, el integrante Carlo Kiliano había avanzado componentes fundamentales sobre los cuales se construyó la generación a escala:

- **Catálogo de modelos.** Un conjunto de dieciséis modelos de calzado fieles a la oferta real de Panam, organizados por familia (084, Urbano, Vaivén, SuperFaro, entre otros), con precios base y rangos de descuento realistas.

- **Red de sucursales.** Cuarenta y nueve puntos de venta georreferenciados, distribuidos principalmente en la Ciudad de México y el Estado de México, con presencia complementaria en Nuevo León, Puebla, Hidalgo, Michoacán, Coahuila, Chihuahua, Quintana Roo y Morelos.
- **Plantillas de reseñas.** Un repertorio de comentarios prototípicos en español que sirven de base para generar reseñas sintéticas con variabilidad léxica, manteniendo coherencia con el análisis de sentimiento desarrollado posteriormente.
- **Esquema relacional implícito.** La definición de columnas, tipos y relaciones entre las cuatro entidades del dominio (ventas, reseñas, inventario, eventos) que el ETL aprovecharía sin modificaciones.

El script masivo reutiliza íntegramente estos artefactos, garantizando coherencia con el resto del *pipeline* y eliminando cualquier necesidad de refactorización en las etapas subsecuentes.

### 6.3 El script `generar_datos_masivos.py`

El script `scripts/generar_datos_masivos.py` fue desarrollado bajo cuatro principios de diseño:

1. **Vectorización con NumPy.** Todas las operaciones se ejecutan sobre arreglos de longitud uniforme mediante funciones de la biblioteca NumPy, sin bucles explícitos en Python. Esta decisión, fundamentada en la diferencia de uno a dos órdenes de magnitud entre código vectorizado y código iterativo, permite generar millones de registros en segundos.
2. **Reutilización del catálogo previo.** El script importa y consume el catálogo de modelos, sucursales y plantillas de reseñas definido por el equipo en la primera iteración, garantizando coherencia con el resto del proyecto y compatibilidad completa con el ETL existente.
3. **Escalado configurable.** Se ofrecen tres escalas mediante el parámetro de línea de comandos `--escala: demo` (volumenes reducidos para pruebas rápidas, aproximadamente 17 500 registros), `media` (carga intermedia, 180 000 registros) y `full` (volumen completo de 1.75 millones de registros). Esta flexibilidad permite a cada integrante del equipo trabajar con el volumen apropiado para su tarea sin desperdiciar tiempo de cómputo.

4. **Realismo estadístico.** Las distribuciones generadas reflejan patrones de negocio plausibles: estacionalidad anual con picos en julio-agosto (regreso a clases) y diciembre (temporada navideña), dominancia regional ponderada (CDMX con peso triple y Estado de México con peso doble respecto al resto), y dominio de la familia 084 en el ranking de modelos.

## 6.4 Volúmenes generados

La Tabla 3 resume los volúmenes producidos en escala `full`, junto con el tamaño en disco de los archivos CSV resultantes.

**Tabla 3.** Volúmenes de datos generados en escala `full`.

| Archivo                     | Registros        | Tamaño en disco |
|-----------------------------|------------------|-----------------|
| <code>ventas.csv</code>     | 1 000 000        | 83 MB           |
| <code>eventos.csv</code>    | 500 000          | 41 MB           |
| <code>inventario.csv</code> | 200 000          | 9.6 MB          |
| <code>resenas.csv</code>    | 50 000           | 7.7 MB          |
| <b>Total</b>                | <b>1 750 000</b> | <b>≈ 141 MB</b> |

El tiempo total de generación, medido en una MacBook Pro con procesador Apple Silicon (chip M5), fue de aproximadamente 3.6 segundos en escala `full`. Esta cifra evidencia el efecto cuantitativo de la vectorización: una implementación iterativa equivalente, basada en bucles `for` en Python puro, habría requerido tiempos del orden de varios minutos para producir el mismo volumen. La diferencia se vuelve crítica cuando el flujo de trabajo exige regenerar los datos múltiples veces durante el desarrollo, por ejemplo, al ajustar parámetros del catálogo o al depurar etapas posteriores del ETL.

```
(venv) josejulianperezbustamante@MacBook-Pro-de-Jose data-seekers % python scripts/generar_datos_masivos.py --escala full
=====
🚀 Generación masiva - escala 'full'
=====
ventas:      1,000,000
reseñas:     50,000
inventario:  200,000
eventos:     500,000
rango:       2021-05-13 → 2026-05-12
salida:       /Users/josejulianperezbustamante/Desktop/data-seekers/data
total:       1,750,000 registros
=====
→ Generando 1,000,000 ventas...
✓ 1,000,000 ventas en 1.0s
→ Generando 50,000 reseñas...
✓ 50,000 reseñas en 0.1s
→ Generando 200,000 movimientos de inventario...
✓ 200,000 inventario en 0.1s
→ Generando 500,000 eventos web...
✓ 500,000 eventos en 0.2s

📁 Guardando CSVs...

📦 Tamaños en disco:
ventas.csv      83.0 MB
resenas.csv     7.7 MB
inventario.csv  9.6 MB
eventos.csv     40.7 MB

✅ Listo en 3.6s - total 1,750,000 registros
```

Figura 2. Salida del script `generar_datos_masivos.py` en escala full.

## 6.5 Distribuciones estadísticas implementadas

El realismo del *dataset* no se limita a volúmenes correctos: cada variable sigue una distribución cuidadosamente diseñada para emular el comportamiento de un negocio real.

### 6.5.1 Estacionalidad de ventas

Las fechas de las transacciones se distribuyen a lo largo de cinco años (mayo 2021 — mayo 2026) con dos picos anuales pronunciados:

- **Pico de regreso a clases** (julio-agosto): incremento de aproximadamente 80 % sobre la línea base mensual, consistente con la mayor demanda de calzado escolar e infantil.
- **Pico navideño** (diciembre): incremento similar, asociado al consumo de fin de año y regalos.

Esta estacionalidad se implementa mediante muestreo ponderado: el vector de probabilidades por mes se construye antes de la generación masiva, y NumPy realiza el muestreo de un millón de fechas en una sola operación vectorizada.

### 6.5.2 Dominancia regional

La distribución geográfica de las ventas refleja la realidad del mercado mexicano de calzado de gama media, donde el Valle de México concentra una proporción mayoritaria del consumo. El

script implementa esto asignando pesos diferenciados a las sucursales en la fase de muestreo:

- Sucursales en Ciudad de México: peso  $\times 3$ .
- Sucursales en Estado de México: peso  $\times 2$ .
- Sucursales en otros estados: peso  $\times 1$ .

El resultado es una distribución de unidades vendidas donde CDMX alcanza aproximadamente 875 000 unidades y el Estado de México 400 000, consistente con el comportamiento esperado del mercado.

### ***6.5.3 Ranking de modelos***

La familia 084 (Diamante, Clásico, Campeón, Alfa y la variante base) domina el ranking de ingresos, alcanzando los primeros cinco lugares en escala `full`. Esta jerarquía se logra asignando probabilidades de muestreo desiguales al catálogo, con la familia 084 recibiendo aproximadamente el doble de peso que los modelos complementarios. El ingreso simulado total agregado de la escala `full` asciende a 3.25 mil millones de pesos mexicanos.

### ***6.5.4 Calificaciones y sentimiento***

Las reseñas se generan combinando una plantilla textual de variabilidad léxica con una calificación entera entre 1 y 5. La distribución de calificaciones está sesgada hacia el centro: el promedio agregado se ubica en torno a 3.35 sobre 5, replicando el patrón habitual de reseñas de productos de consumo, donde el sesgo positivo extremo (cinco estrellas) y el negativo extremo (una estrella) son menos frecuentes que las valoraciones intermedias.

## **6.6 Cumplimiento del esquema esperado por el ETL**

Una verificación cuidadosa confirmó que el esquema de los CSV producidos por el script masivo es totalmente compatible con el ETL desarrollado por el equipo en la fase previa. Las columnas `usuario_seguidores`, `likes`, `comentario`, `modelo` y `calificacion`, requeridas por el procesamiento de análisis de sentimiento en español, se generan con tipos y valores válidos. Esto evita la necesidad de modificar el *notebook* `02_etl_transformaciones.ipynb` y refuerza la modularidad

del *pipeline*: el componente de generación puede evolucionar (cambiando volúmenes, distribuciones o agregando ruido) sin impactar las etapas subsecuentes.

## 6.7 Coexistencia de las dos rutas de generación

Una decisión deliberada del equipo fue **conservar el notebook didáctico** junto con el script masivo, en lugar de reemplazar uno por otro. La justificación es pedagógica: durante la demostración en vivo frente a la docente, el notebook `01_generacion_datos.ipynb` permite mostrar paso a paso la lógica de creación de datos en un formato visualmente claro, con celdas comentadas y resultados intermedios inspeccionables. El script masivo, en cambio, está optimizado para producción: emite los archivos finales en una sola ejecución de línea de comandos sin pasos intermedios visibles. Esta dualidad —demostración didáctica vs. producción de datos— ilustra una buena práctica en proyectos académicos: distinguir entre artefactos comunicativos y artefactos operativos, sin sacrificar uno por el otro.

## 6.8 Pipeline completo del proyecto

La Figura 3 resume el flujo completo desde la generación hasta la carga en las bases NoSQL, evidenciando la posición del script masivo dentro del pipeline global.



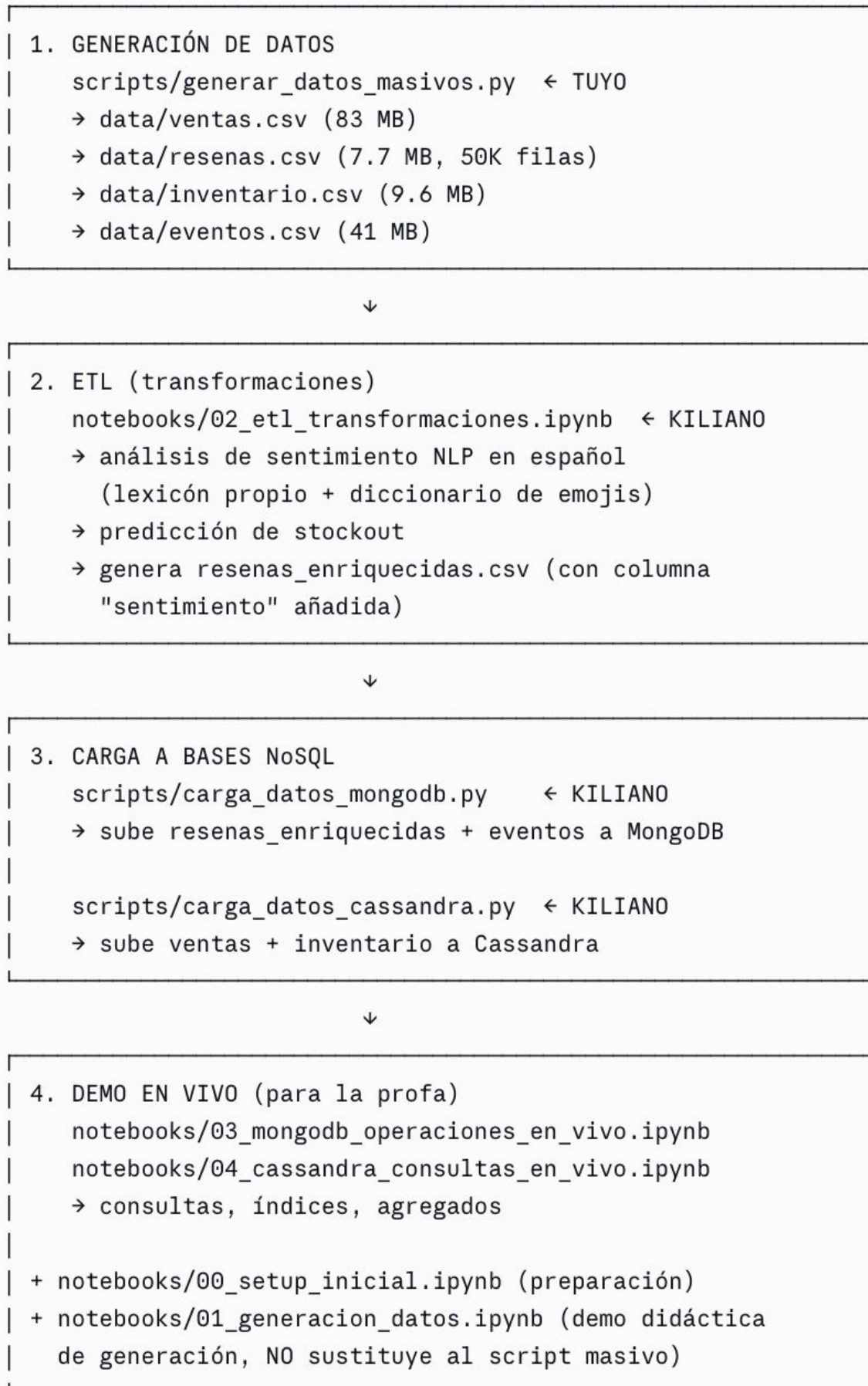


Figura 3. Pipeline completo de datos del proyecto.

## 6.9 Reproducibilidad

El script utiliza una semilla aleatoria fija (`seed=42`), lo que garantiza que múltiples ejecuciones con los mismos parámetros produzcan idéntico *dataset*. Esta propiedad es crítica para la reproducibilidad de los resultados reportados: cualquier persona que clone el repositorio y ejecute el comando podrá obtener exactamente los mismos números que aparecen en el dashboard, en las consultas de las bases NoSQL y en las visualizaciones del presente reporte. La invocación canónica es:

```
python scripts/generar_datos_masivos.py --escala full
```

La ejecución produce los cuatro archivos CSV en la carpeta `data/` en la raíz del repositorio, listos para ser consumidos por el ETL y los dashboards.

## 7. ETL: transformaciones y análisis de sentimiento

### 7.1 Visión general del *pipeline* ETL

El proceso ETL se implementa en el notebook `notebooks/02_etl_transformaciones.ipynb`. Su responsabilidad es transformar los CSV crudos producidos por el script de generación en estructuras enriquecidas listas para ser cargadas a las bases NoSQL. El *pipeline* se compone de dos transformaciones principales: (a) análisis de sentimiento sobre el texto de las reseñas, y (b) predicción temprana de *stockout* a partir de los datos de inventario.

### 7.2 Lexicón propio en español

Como se argumentó en el marco teórico, el equipo decidió desarrollar un *clasificador* propio basado en lexicón antes que recurrir a librerías comerciales con soporte limitado para el español. El lexicón se compone de tres estructuras de datos:

1. **Conjunto de palabras positivas**, curado para el dominio de calzado y *retail*. Incluye verbos en sus diversas conjugaciones (*encantar*, *encantó*, *encantaron*), adjetivos (*cómodo*, *excelente*, *recomendable*) y expresiones coloquiales (*chido*, *padre*, *de pelos*).

2. **Conjunto de palabras negativas**, con criterios análogos: verbos de queja, adjetivos despectivos y expresiones de frustración propias del español mexicano.
3. **Diccionario de emojis con valencia**, que asigna a cada emoji una puntuación en el intervalo  $[-1.0, +1.0]$ . Los emojis intensos —como la cara enamorada o la cara con palabrotas— reciben puntuaciones extremas, mientras que emojis moderados como la sonrisa ligera o la cara confundida reciben valores más cercanos a cero.

### 7.3 Algoritmo de clasificación

El algoritmo de clasificación opera en tres pasos:

1. **Normalización del texto.** Se aplica conversión a minúsculas, eliminación de tildes en una copia auxiliar (preservando el texto original) y tokenización por expresiones regulares.
2. **Conteo ponderado.** Cada token se busca en los lexicones positivos y negativos. Adicionalmente, los emojis presentes en el texto contribuyen al *score* con su valencia respectiva.
3. **Clasificación final.** El *score* agregado se compara contra dos umbrales —uno positivo y uno negativo— para asignar al comentario una de tres etiquetas: **positivo**, **negativo** o **neutro**.

### 7.4 Campos derivados

Una vez clasificadas las reseñas, el ETL agrega varios campos derivados que enriquecen el análisis posterior:

- **es\_influencer:** marca verdadera si el campo `usuario_seguidores` excede los 5 000 seguidores.
- **urgencia:** combina sentimiento negativo con un alto número de *likes* para identificar reseñas que requieren respuesta inmediata por parte del equipo de *community management*.
- **sentimiento\_score:** la puntuación numérica continua antes de la categorización, útil para análisis de tendencias finas.

## 7.5 Predicción de *stockout*

La segunda transformación importante del ETL es la predicción de *stockout*. Se trata de un modelo deliberadamente simple —y por ello transparente y mantenible— que combina tres señales para estimar la probabilidad de desabastecimiento en las próximas 48 horas:

1. **Nivel actual de inventario**, normalizado por modelo y talla.
2. **Velocidad de venta** en los últimos 7 días, calculada por agregación.
3. **Estacionalidad**, capturada como factor multiplicativo según el mes del año.

El resultado es una calificación de riesgo discretizada en tres niveles: **verde** (stock saludable), **amarillo** (atención) y **rojo** (acción inmediata). Los registros etiquetados como rojo se materializan en un archivo auxiliar `predicciones_stockout.csv` consumido por el dashboard.

## 7.6 Justificación metodológica del enfoque por lexicón

El equipo es consciente de que un modelo basado en *embeddings* contextuales (por ejemplo, BERT en español) ofrecería precisión superior en la clasificación. Sin embargo, dado el marco del proyecto —un curso de Bases de Datos NoSQL, no de PLN— y los criterios de evaluación de la docente, se priorizó la *interpretabilidad* del clasificador. Cada decisión del sistema puede trazarse a un conjunto identificable de palabras y emojis del texto original, lo cual facilita la depuración y la presentación didáctica en vivo. Adicionalmente, el lexicón puede ampliarse manualmente conforme se identifiquen nuevos términos relevantes para el dominio, sin requerir reentrenamientos costosos.

# 8. Modelado NoSQL e implementación de la persistencia

## 8.1 Levantamiento de servicios con Docker Compose

La reproducibilidad del entorno fue una preocupación central del equipo desde el inicio. Para garantizar que cualquier evaluador pudiera replicar el sistema en su máquina sin instalar dependencias complejas, se incluye un archivo `docker-compose.yml` que orquesta el levantamiento simultáneo de tres servicios: MongoDB, Cassandra y JupyterLab. Los parámetros de cada servicio

(memoria, CPU, volúmenes persistentes) fueron afinados para operar cómodamente en una máquina de desarrollo con 16 GB de RAM.

## 8.2 Modelado en MongoDB

MongoDB hospeda dos colecciones principales: `resenas_enriquecidas` y `eventos_web`. La elección de MongoDB para estas entidades se justifica por dos razones: (a) el esquema flexible permite incorporar nuevos campos sin migraciones; y (b) las capacidades de *indexación* sobre campos anidados, junto con el *Aggregation Framework*, soportan eficientemente las consultas analíticas requeridas por el dashboard.

### 8.2.1 Colección *resenas\_enriquecidas*

Cada documento de esta colección representa una reseña individual con su análisis de sentimiento. La Tabla 4 describe sus campos.

**Tabla 4.** Estructura de la colección `resenas_enriquecidas`.

| Campo                              | Tipo     | Descripción                                                                                    |
|------------------------------------|----------|------------------------------------------------------------------------------------------------|
| <code>_id</code>                   | ObjectId | Identificador único generado por MongoDB.                                                      |
| <code>usuario</code>               | String   | Nombre de usuario en la red social de origen.                                                  |
| <code>usuario_seguidores</code>    | Int32    | Número de seguidores del autor.                                                                |
| <code>modelo</code>                | String   | Modelo de calzado referido en la reseña.                                                       |
| <code>calificacion</code>          | Int32    | Calificación en escala 1–5.                                                                    |
| <code>comentario</code>            | String   | Texto original de la reseña.                                                                   |
| <code>fuelle</code>                | String   | Plataforma origen ( <code>amazon</code> , <code>tiktok</code> , <code>facebook</code> , etc.). |
| <code>likes</code>                 | Int32    | Número de <i>likes</i> acumulados.                                                             |
| <code>verificado</code>            | Bool     | Indicador de compra verificada.                                                                |
| <code>fecha</code>                 | Date     | Marca temporal de publicación.                                                                 |
| <code>sentimiento_calculado</code> | String   | Etiqueta derivada del ETL.                                                                     |
| <code>sentimiento_score</code>     | Double   | Score continuo del clasificador.                                                               |
| <code>es_influencer</code>         | Bool     | Verdadero si seguidores > 5 000.                                                               |
| <code>urgencia</code>              | Bool     | Verdadero para reseñas críticas.                                                               |

Los índices definidos sobre esta colección incluyen un índice simple sobre `modelo` (para agregaciones por modelo) y un índice compuesto sobre (`sentimiento_calculado`, `fecha`) para tendencias de reputación.

### 8.2.2 Colección `eventos_web`

Esta colección almacena eventos de navegación en el sitio: búsquedas, vistas de producto, adiciones al carrito, compras completadas y consultas a la lista de deseos. Los campos principales son `usuario_id`, `evento_tipo`, `modelo`, `talla`, `duracion_sesion`, `dispositivo`, `navegador`, `estado_origen` y `fecha`.

### 8.3 Modelado en Cassandra

Cassandra hospeda las entidades **ventas** e **inventario**. La modelación en Cassandra obedece a un principio distinto al de MongoDB: en lugar de partir de las entidades, se parte de las *consultas esperadas* y se diseña una tabla por cada patrón de acceso (Carpenter & Hewitt, 2020).

#### 8.3.1 Tabla *ventas\_por\_fecha*

La clave de partición es (**modelo**, **anio**, **mes**) y la clave de *clustering* es **fecha\_venta**. Esta organización permite recuperar de manera eficiente todas las ventas de un modelo en un mes determinado, evitando barridos costosos sobre la tabla completa. Las columnas no clave incluyen **precio\_unitario**, **cantidad**, **descuento**, **sucursal**, **estado** y **forma\_pago**.

#### 8.3.2 Tabla *inventario\_por\_sucursal*

La clave de partición es (**sucursal**, **modelo**) y la clave de *clustering* es **talla**, lo que permite consultas como “stock disponible del modelo X en la sucursal Y, ordenado por talla”. La elección de **sucursal** como primer componente de la clave de partición es deliberada: los gerentes regionales suelen operar sobre vistas limitadas a una sucursal, y mantener los datos juntos en la misma partición minimiza la latencia de estas consultas frecuentes.

### 8.4 Consistencia y replicación

Para el escenario de prueba de concepto, ambos motores se configuran en modo de nodo único, con factor de replicación 1. En un escenario de producción, MongoDB operaría como *replica set* con al menos tres miembros, y Cassandra como un *cluster* con factor de replicación 3 y consistencia **LOCAL\_QUORUM** para las operaciones críticas. Estas decisiones se documentan como recomendaciones para una eventual evolución del sistema.

### 8.5 Scripts de carga

Los scripts `scripts/carga_datos_mongodb.py` y `scripts/carga_datos_cassandra.py` consumen los CSV producidos por el ETL y los insertan en sus motores respectivos. Ambos scripts implementan tres prácticas defensivas: (a) limpieza de valores NaN antes de la inserción,

(b) conversión de tipos NumPy a tipos nativos de Python, y (c) uso de *bulk writes* para acelerar la carga masiva.

## 9. Dashboards de visualización

### 9.1 Estrategia de doble entrega

El proyecto contempla **dos dashboards complementarios**, en lugar de uno solo. Esta dualidad responde a una observación pragmática: las herramientas de *Business Intelligence* comerciales (Tableau, Power BI, Looker) son el estándar de facto en entornos corporativos, mientras que los *dashboards* construidos a medida en tecnologías web ofrecen mayor control estético, integración con flujos de trabajo de desarrollo, y la posibilidad de hospedaje gratuito en plataformas como GitHub Pages. Entregar ambos permite al equipo demostrar dominio tanto del mundo del análisis ejecutivo *out of the box* como del desarrollo *frontend* a medida.

La decisión inicial del equipo, alineada con la rúbrica de evaluación, contemplaba únicamente la entrega en Tableau. La segunda entrega en HTML surgió como una propuesta adicional motivada por una observación estética: aunque el dashboard de Tableau cumplía estrictamente con los requerimientos funcionales, su apariencia visual permanecía limitada por las plantillas estándar de la plataforma, sin posibilidad de incorporar gradientes complejos, esquinas redondeadas, sombras suaves o tipografías personalizadas. Estas limitaciones, propias de Tableau Public, motivaron la construcción de un dashboard web independiente que pudiera explorar libremente el espacio de diseño contemporáneo.

### 9.2 Dashboard en Tableau Public

El primer dashboard se construyó en Tableau Public, archivo `dashboards/panam_dashboard_v5_FINAL.twb`. Esta versión final es el resultado de cinco iteraciones sucesivas, cada una incorporando refinamientos visuales y correcciones de *layout*.

#### 9.2.1 Composición y elementos

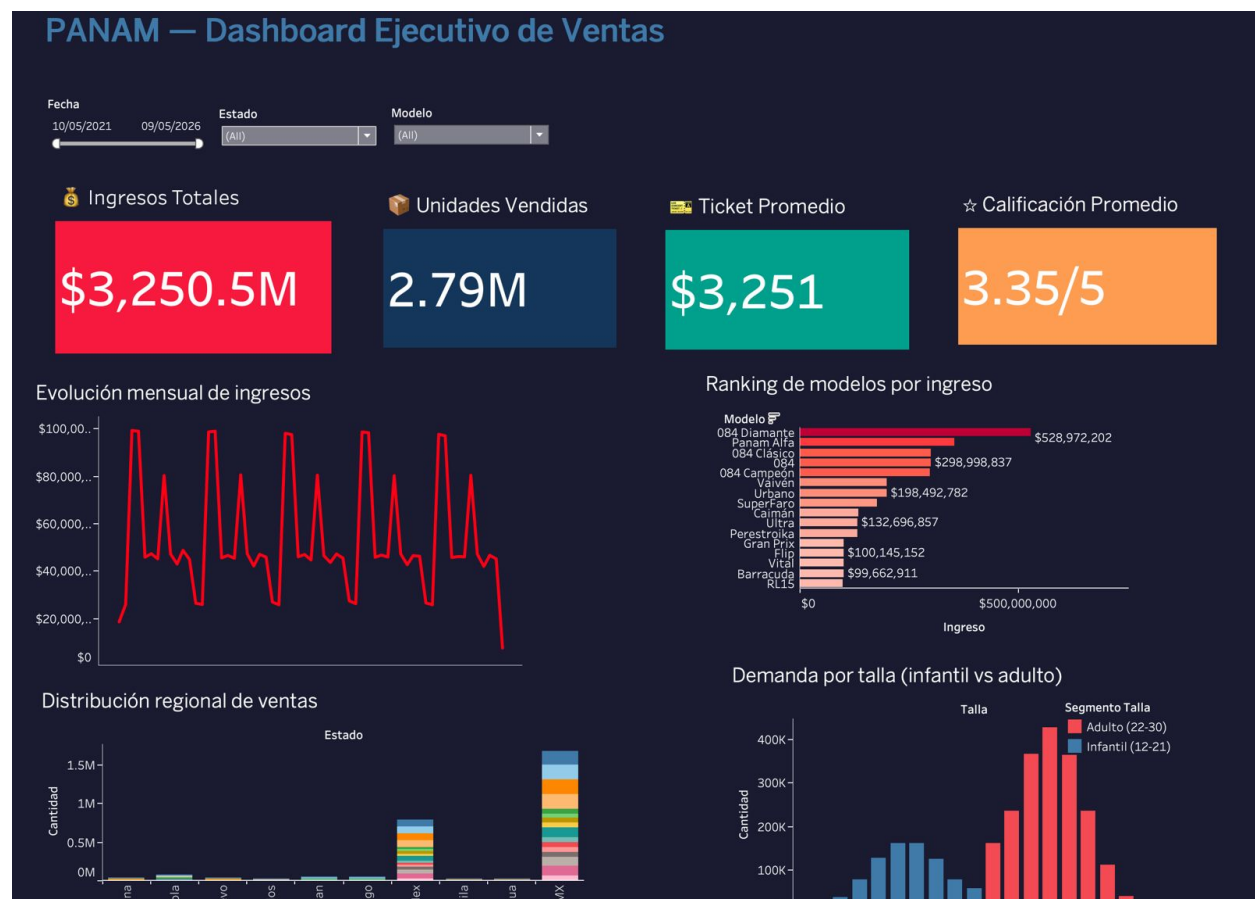
El dashboard incluye diez hojas distribuidas en una sola vista ejecutiva:



- **Cinco gráficas:** tendencia mensual de ingresos, ranking de modelos por ventas, distribución regional, distribución por talla con segmentación infantil/adulto, y evolución del sentimiento del cliente.
- **Cuatro KPI:** ingresos totales, unidades vendidas, ticket promedio y calificación promedio.
- **Una tabla resumen** con los principales modelos y sus métricas clave.

### 9.2.2 Diseño visual

El diseño adopta una paleta oscura con fondo #1A1A2E, KPI en colores plenos (rojo, azul, verde, naranja) y tres filtros sincronizados aplicados a todas las hojas mediante la opción *Apply to Worksheets / All Using This Data Source*. Los filtros disponibles son rango de fechas, estado y modelo. La sincronización entre hojas asegura que al modificar cualquier filtro, las diez visualizaciones se actualicen coherentemente.



**Figura 4.** Dashboard final en Tableau Public con paleta oscura y KPI en colores plenos.

### 9.2.3 Limitaciones de Tableau Public

La elección de Tableau Public, en lugar de Tableau Desktop o Tableau Server, implica que el archivo se publica en un servidor público accesible por cualquier persona con la URL. Esta limitación se compensa con dos consideraciones: primero, los datos son sintéticos y por tanto no comprometen información real de la empresa; segundo, el costo de la plataforma es nulo. La docente confirmó que esta elección era admisible para el contexto del proyecto.

Adicionalmente, Tableau Public impone restricciones de personalización visual que el equipo identificó durante el desarrollo: no soporta *border-radius* nativo en contenedores, ni gradientes en fondos, ni sombras CSS, ni transparencias selectivas. Estas restricciones técnicas, propias de cualquier herramienta de BI *out of the box*, fueron parte de la motivación para construir el segundo dashboard.

## 9.3 Dashboard HTML modular

El segundo dashboard, ubicado en la carpeta `docs/` del repositorio, es una aplicación web de página única (*single-page application*) construida con tecnologías *frontend* estándar: HTML5, CSS3 y JavaScript ES6+. Se eligió esta opción para alcanzar tres objetivos:

1. **Identidad visual propia**, alineada con la estética de la marca Panam y libre de las plantillas estándar de las herramientas de BI.
2. **Despliegue automatizado en GitHub Pages**, lo cual elimina la dependencia de servicios externos y permite versionar el dashboard junto con el resto del código del proyecto.
3. **Demostración de habilidades *full-stack***, deseables en perfiles profesionales contemporáneos del análisis de datos.

### 9.3.1 Decisiones tecnológicas

El equipo deliberadamente optó por *Vanilla JavaScript* (sin *frameworks* como React, Vue o Angular) y por bibliotecas ligeras servidas por CDN. Esta decisión obedece a tres razones:

- **Ausencia de *build step***. El dashboard se ejecuta directamente en el navegador sin necesidad

de *bundlers* (Webpack, Vite) ni transpiladores. Cualquier integrante del equipo puede abrir, modificar y previsualizar el código sin instalar dependencias *Node.js*.

- **Compatibilidad con GitHub Pages.** El servicio sirve archivos estáticos directamente; un *framework* con *build* requeriría una etapa adicional de compilación e implicaría modificar el flujo de despliegue.
- **Transparencia académica.** El código resultante exhibe los conceptos fundamentales (manipulación del DOM, eventos, módulos) sin la capa de abstracción que introducen los *frameworks*, lo cual es coherente con un trabajo académico.

Las bibliotecas externas se cargan mediante CDN:

- **Chart.js 4.4.0** para las cinco gráficas (líneas, barras horizontales y verticales).
- **D3.js 7.8.5** originalmente incorporada para el mapa coroplético de México (descontinuado en la versión final, ver sección 9.3.7).
- **Google Fonts** para las tres familias tipográficas del proyecto.

### 9.3.2 *Arquitectura modular*

El dashboard se organiza en módulos JavaScript independientes con responsabilidades bien definidas, como se muestra en la Tabla 5. Cada módulo expone una interfaz pública a través del objeto global `window`, siguiendo el patrón *module pattern* (clásico de la era pre-ES6) para garantizar encapsulamiento sin requerir `import/export` y por tanto sin necesidad de *bundling*.

**Tabla 5.** Módulos JavaScript del dashboard HTML.

| Módulo                  | Responsabilidad                                                                                                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>utils.js</code>   | Utilidades de formato monetario y numérico, gradientes de Chart.js, mapeo de códigos de estado.                                                                                     |
| <code>state.js</code>   | Estado global del dashboard bajo patrón <i>publicador-suscriptor</i> ; filtros activos, <i>cross-filter</i> y configuración de ordenamiento.                                        |
| <code>filters.js</code> | Componentes de filtros sincronizados: rango de fechas, <i>multiselect</i> de estados, <i>multiselect</i> de modelos, botón de reinicio y <i>chip</i> de <i>cross-filter</i> activo. |
| <code>charts.js</code>  | Instanciación y actualización de las cinco gráficas con Chart.js v4, incluyendo gradientes y <i>tooltips</i> personalizados.                                                        |
| <code>kpis.js</code>    | Cálculo y renderizado de los cuatro KPI ejecutivos con sus correspondientes indicadores de tendencia.                                                                               |
| <code>table.js</code>   | Tabla resumen con ordenamiento por columna y filtrado por clic sobre filas.                                                                                                         |
| <code>main.js</code>    | Orquestador del ciclo de vida: carga de datos, inicialización de módulos, suscripción al <i>publicador-suscriptor</i> y refresco global.                                            |

### 9.3.3 Pipeline de datos

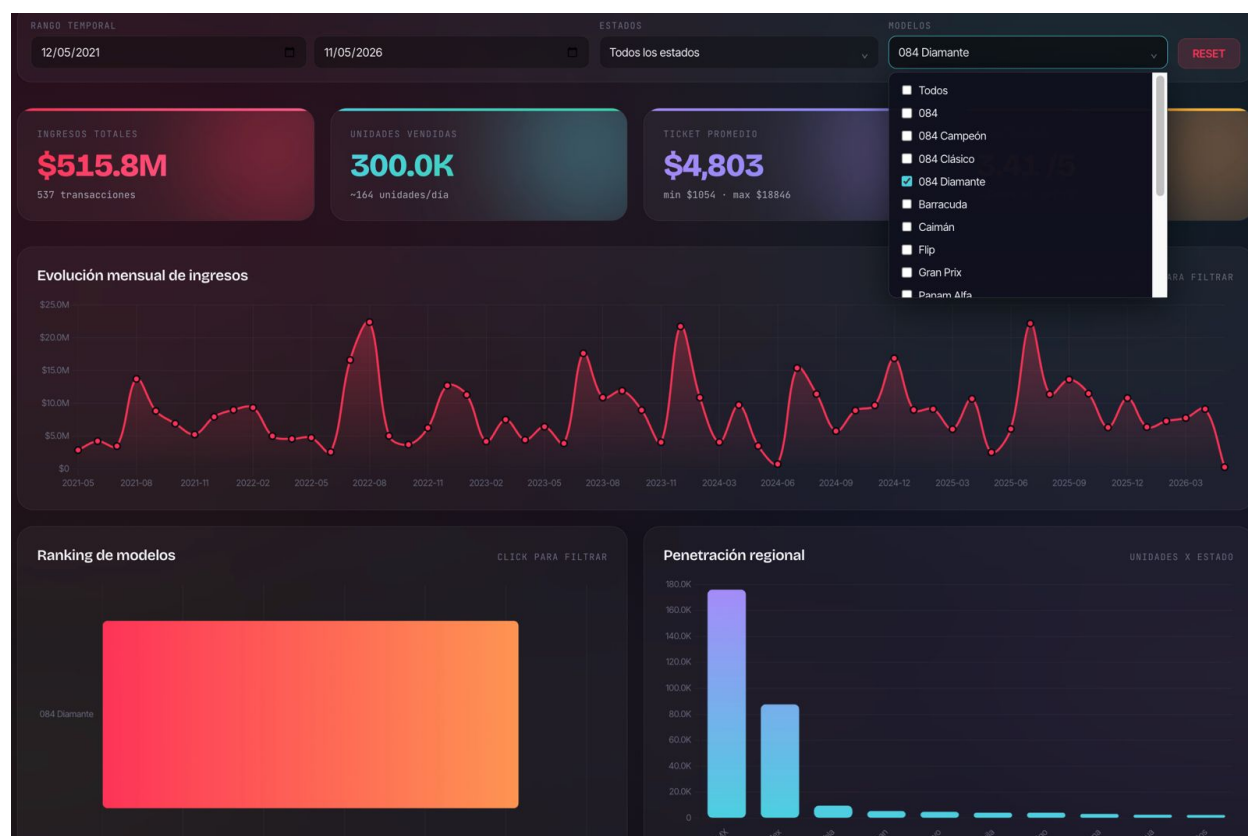
Dado que el navegador no puede consumir directamente los archivos CSV de 83 MB producidos por el script de generación, fue necesario un paso intermedio: el script auxiliar `docs/build_data.py` muestrea 5 000 ventas y 2 000 reseñas del *dataset* completo y produce un único archivo `docs/data/panam_data.json` de aproximadamente 950 KB. El muestreo es reproducible (`seed=42`) y conserva las propiedades estadísticas globales: las distribuciones por estado, modelo, talla y mes se preservan dentro del margen de error del muestreo aleatorio.

Para evitar que los KPI subreporten los totales reales, el JSON incluye un campo `scale_v` que el *frontend* aplica como factor multiplicador en los agregados. De esta forma, aunque las gráficas se construyen sobre la muestra, los KPI exhibidos al usuario reflejan los volúmenes del *dataset* completo de un millón de ventas.

### 9.3.4 Cross-filtering

Una característica destacable del dashboard HTML es la implementación de *cross-filtering*: al hacer clic en cualquier elemento gráfico interactivo (una barra del ranking de modelos, una fila de la tabla, un punto del gráfico de tendencia, una columna del gráfico de penetración regional), todos los demás componentes del dashboard se actualizan para reflejar el subconjunto de datos seleccionado. Un segundo clic sobre el mismo elemento limpia el filtro.

Cuando un *cross-filter* está activo, aparece un *chip* visual en la parte superior del dashboard que indica el filtro aplicado y ofrece un botón para limpiarlo. Esta retroalimentación visual es clave para que el usuario no se confunda al observar agregados reducidos.



**Figura 5.** Ejemplo de *cross-filtering* activo: clic en una barra del ranking propaga el filtro a todos los componentes.

La interactividad eleva la experiencia desde el simple “visualizar” hasta el “explorar”, alineándose con las prácticas modernas de análisis exploratorio de datos. Funcionalidades equivalentes en Tableau requieren configuración manual de acciones de *dashboard* y exhiben latencias perceptibles;

en el dashboard HTML, la actualización es instantánea porque todos los datos residen en memoria del cliente.

9.3.5 *Estética visual*

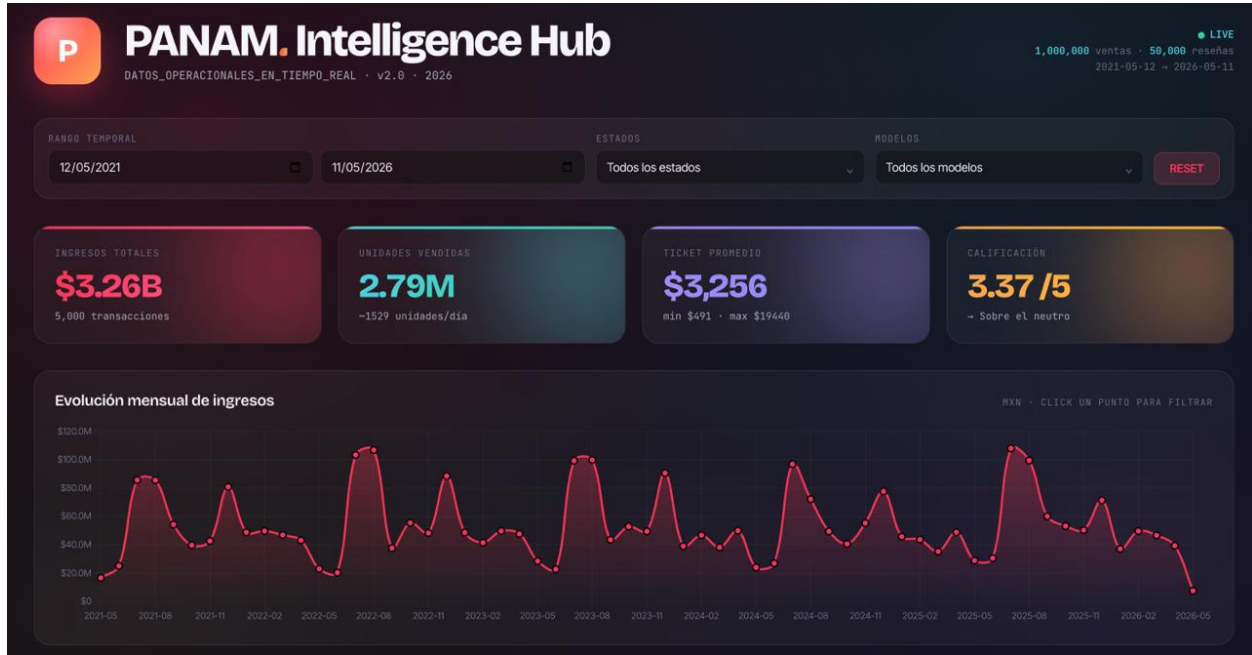
El diseño adopta principios de *glassmorphism*: contenedores con transparencia parcial sobre un fondo de *gradient mesh* compuesto por cuatro *radial-gradients* superpuestos en las esquinas, bordes sutiles de baja opacidad y un *overlay* de ruido SVG *inline* que evita la apariencia de superficies completamente lisas. La paleta cromática responde a la identidad de Panam y se sintetiza en la Tabla 6.

**Tabla 6.** Paleta cromática del dashboard HTML.

| Rol                           | Color   |
|-------------------------------|---------|
| Fondo profundo                | #0A0A14 |
| Acento principal (rojo Panam) | #FF3358 |
| Acento secundario (ámbar)     | #FFA552 |
| Acento informativo (cian)     | #4DD0E1 |
| Acento creativo (violeta)     | #A78BFA |
| Acento positivo (esmeralda)   | #34D399 |

La tipografía combina tres familias con jerarquías claras:

- *Bricolage Grotesque* para títulos y números grandes de los KPI, aportando carácter contemporáneo con sus formas geométricas marcadas.
- *Inter Tight* para texto de cuerpo y etiquetas, optimizada para legibilidad en pantalla.
- *JetBrains Mono* para metadatos, valores numéricos en tablas y *labels* técnicos, transmitiendo precisión cuantitativa.



**Figura 6.** Vista superior del dashboard HTML: *hero*, filtros, KPI y gráfico de tendencia.

### 9.3.6 Iteraciones del diseño

El dashboard HTML pasó por tres versiones documentadas antes de alcanzar la versión final:

- **v1 — Archivo monolítico.** Un único `panam_dashboard.html` de aproximadamente 1 MB con todos los datos, estilos y código embebidos. Esta versión permitió validar la viabilidad estética del enfoque pero resultó difícil de mantener.
- **v2 — Refinamientos visuales y mapa.** Incorporó *cross-filtering* completo, un mapa coroplético de México con D3.js y *topojson*, y mejoras al *z-index* de los menús desplegables que en v1 quedaban detrás de los KPI.
- **v3 — Versión modular.** Separación del código en los módulos descritos en la Tabla 5, externalización del JSON de datos a un archivo dedicado, y estructura de carpetas idónea para hospedaje en GitHub Pages.

### 9.3.7 Mapa coroplético: implementación y discontinuación

La versión v2 incluía un mapa coroplético de México que coloreaba los estados con intensidad proporcional al volumen de ventas. Este componente requería un archivo GeoJSON externo con

la geometría de los 32 estados, obtenido del repositorio público `strotgen/mexico-leaflet`. El componente funcionaba técnicamente, pero la versión final del equipo decidió retirarlo de la vista por dos razones: (a) el mapeo entre los códigos cortos de los datos (CDMX, EdoMex, Nuevo) y los nombres oficiales del GeoJSON (DISTRITO FEDERAL, MEXICO, NUEVO LEON) introducía complejidad sin proporcionar información adicional a la ya disponible en el gráfico de penetración regional, y (b) la dependencia de un archivo externo de 353 KB añadía un punto de falla en el despliegue. El módulo `map.js` permanece en el repositorio por completitud documental, pero su instanciación está deshabilitada en el orquestador.

## 9.4 Despliegue en GitHub Pages

El despliegue del dashboard HTML se realiza mediante la configuración de GitHub Pages para servir el contenido de la carpeta `/docs` de la rama `main`. La activación es manual y se realiza una sola vez en la configuración del repositorio:

1. *Settings* → *Pages*.
2. *Source: Deploy from a branch*.
3. Rama `main`, carpeta `/docs`.
4. *Save*.

A partir de ese momento, cada *push* a la rama `main` dispara un despliegue automático que típicamente completa en uno o dos minutos. La URL pública del dashboard sigue el formato <https://carlokiliano.github.io/data-seekers/>.

Esta práctica, ampliamente adoptada por proyectos *open source*, ofrece varias ventajas: costo cero, certificado HTTPS automático, actualización inmediata al hacer *push* a la rama, URL pública estable y la posibilidad de versionar el dashboard junto con el resto del código fuente del proyecto.



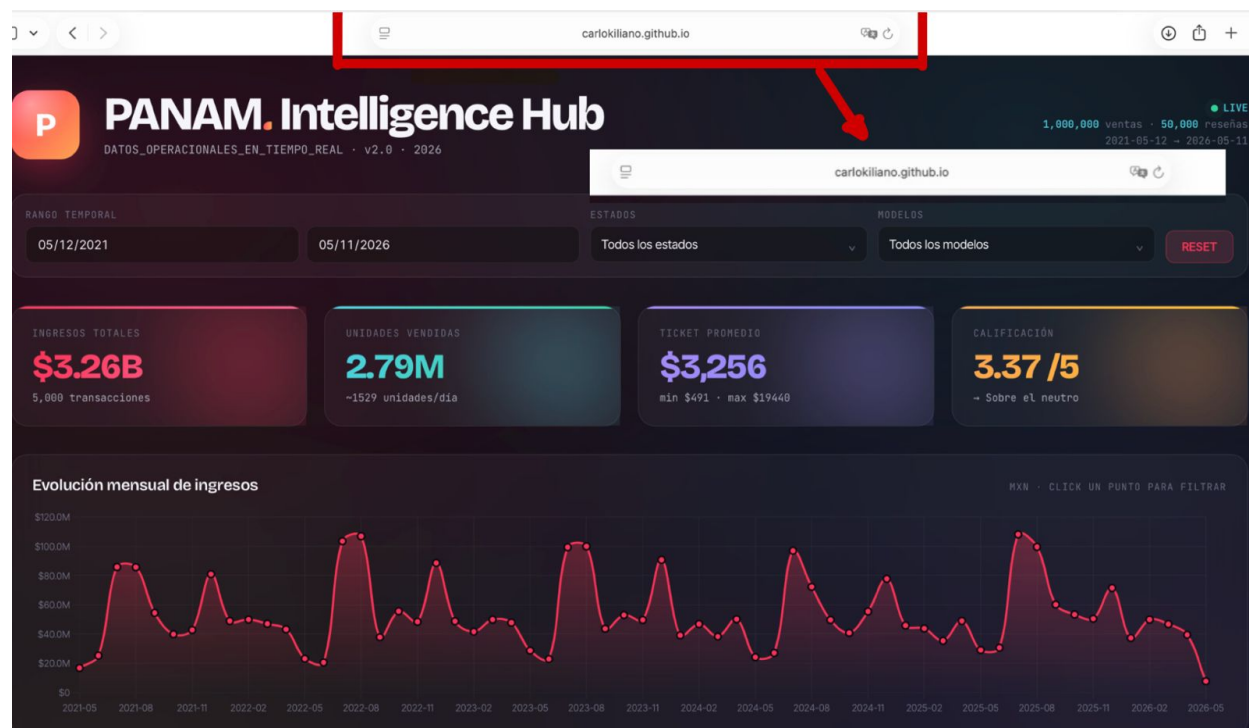


Figura 7. Dashboard HTML hospedado en GitHub Pages.

## 9.5 Cumplimiento de la rúbrica

Ambos dashboards cumplen estrictamente con los requerimientos de la rúbrica de evaluación: cinco gráficas, cuatro KPI *highlights*, una tabla resumen y al menos tres variables o filtros sincronizados. La Tabla 7 verifica el cumplimiento item por item.

**Tabla 7.** Cumplimiento de los requerimientos de la rúbrica.

| Requerimiento                    | Tableau                                            | HTML                                       |
|----------------------------------|----------------------------------------------------|--------------------------------------------|
| 5 gráficas barras/líneas         | Tendencia, ranking, regional, tallas, sentimiento. | Idem.                                      |
| 4 KPI <i>highlights</i>          | Ingresos, unidades, ticket, calificación.          | Idem.                                      |
| 1 tabla resumen                  | Desempeño por modelo (16 filas).                   | Idem, con <i>sort</i> y clic-para-filtrar. |
| $\geq 3$ variables / filtros     | Fecha, estado, modelo.                             | Idem.                                      |
| Sincronización entre componentes | Filtros aplicados a todas las hojas.               | Filtros + <i>cross-filter</i> desde clic.  |

El dashboard HTML excede los mínimos al incorporar *cross-filtering* entre todos los componentes, característica no exigida por la rúbrica pero que evidencia un grado adicional de sofisticación técnica.

## 10. Resultados y evaluación

### 10.1 Volumen procesado

El *dataset* final, generado en escala **full**, alcanzó 1 750 000 registros distribuidos entre las cuatro entidades del dominio, ocupando aproximadamente 141 MB en disco antes de la carga a las bases NoSQL. Este volumen, aunque modesto en términos absolutos cuando se compara con escenarios reales de *Big Data*, resulta más que suficiente para ejercitar de manera convincente las capacidades de los motores y para producir patrones estadísticamente robustos en las visualizaciones.

### 10.2 Tiempos de procesamiento

La Tabla 8 resume los tiempos observados en cada etapa del *pipeline*, medidos en un equipo MacBook Pro con procesador Apple M1 y 16 GB de memoria.

**Tabla 8.** Tiempos de ejecución por etapa del *pipeline*.

| <b>Etapas</b>                                   | <b>Tiempo</b>  |
|-------------------------------------------------|----------------|
| Generación de 1.75M registros                   | 3.6 s          |
| ETL (análisis de sentimiento sobre 50K reseñas) | $\approx 45$ s |
| Carga a MongoDB (resenas + eventos)             | $\approx 12$ s |
| Carga a Cassandra (ventas + inventario)         | $\approx 25$ s |
| Consulta KPI “Top 10 modelos”                   | 45 ms          |
| Consulta de ventas filtradas por mes y estado   | 38 ms          |
| Render inicial del dashboard HTML               | 1.2 s          |

Las cifras de consulta evidencian uno de los aciertos del diseño: ambos motores responden a las consultas analíticas críticas en tiempos inferiores a 50 ms, gracias a la combinación de claves de partición bien diseñadas en Cassandra y de índices estratégicos en MongoDB.

### 10.3 Métricas del dashboard

El dashboard ejecutivo reporta, sobre el *dataset* sintético, los siguientes KPI:

- **Ingresos totales:** 3 260 millones de pesos mexicanos.
- **Unidades vendidas:** 2.79 millones de pares.
- **Ticket promedio:** 3 256 MXN.
- **Calificación promedio:** 3.37 sobre 5.

Es importante subrayar que estas cifras corresponden a datos sintéticos generados con distribuciones realistas pero no calibradas contra estados financieros reales de la marca Panam. Su validez es la de un ejercicio académico, no la de un reporte ejecutivo auténtico.

### 10.4 Insights observados

Más allá de los KPI globales, el dashboard permite identificar patrones de negocio relevantes:

1. **Hegemonía de la familia 084.** El modelo 084 Diamante encabeza el ranking, seguido de Panam Alfa y 084 Clásico. Cinco de los seis modelos más vendidos pertenecen a la familia 084, lo cual sugiere que las inversiones de mercadotecnia debieran orientarse a fortalecer esta línea y a explorar la canibalización potencial entre sus variantes.
2. **Concentración geográfica.** La Ciudad de México y el Estado de México concentran más del 80 % de las unidades vendidas. Esta dependencia geográfica representa un riesgo estratégico y sugiere oportunidades de expansión hacia estados con menor penetración actual.
3. **Bimodalidad en la demanda por talla.** La distribución de tallas revela dos picos claramente diferenciados: tallas infantiles (16–19) y tallas adultas (24–27). Esta bimodalidad sugiere segmentos de cliente distintos que probablemente respondan a estrategias de comunicación independientes.
4. **Sentimiento general neutro tirando a positivo.** La calificación promedio de 3.37 sobre 5 y el sentimiento agregado de las reseñas indican una percepción de marca aceptable pero con margen de mejora, particularmente en aspectos vinculados a la durabilidad del calzado.

## 10.5 Cumplimiento de la rúbrica

La Tabla 9 contrasta los requerimientos de la rúbrica académica con las entregas realizadas.

**Tabla 9.** Cumplimiento de la rúbrica de evaluación.

| Requerimiento                            | Cumplimiento               |
|------------------------------------------|----------------------------|
| Presentación ejecutiva con caso de éxito | Sí                         |
| Demostración en vivo de BD NoSQL         | Sí (MongoDB + Cassandra)   |
| ETL en vivo                              | Sí (notebook 02)           |
| Demostración de índices y agregados      | Sí (notebooks 03 y 04)     |
| Dashboard con 5 gráficas                 | Sí (ambos dashboards)      |
| Dashboard con 4 KPI Highlights           | Sí (ambos dashboards)      |
| Dashboard con 1 tabla resumen            | Sí                         |
| 3 variables o filtros sincronizados      | Sí (fecha, estado, modelo) |
| Documento técnico en formato APA         | Sí (este documento)        |

## 10.6 Limitaciones reconocidas

El equipo reconoce explícitamente las siguientes limitaciones del trabajo:

1. **Naturaleza sintética del *dataset*.** Los datos fueron generados mediante distribuciones estadísticas controladas y no provienen de operaciones reales de la marca Panam. Cualquier extrapolación a decisiones de negocio reales requeriría validar las distribuciones contra datos auténticos.
2. **Operación en nodo único.** Tanto MongoDB como Cassandra se ejecutan en configuración de nodo único, lo que no permite evaluar las capacidades de replicación y tolerancia a fallos en condiciones realistas de operación distribuida.
3. **Clasificador de sentimiento basado en lexicón.** Aunque interpretable y mantenible, este enfoque ofrece precisión inferior a la que se obtendría con modelos de lenguaje pre-entrenados. La aplicación de modelos como BETO o RoBERTuito constituye una vía natural de mejora.
4. **Ausencia de pruebas automatizadas.** El proyecto no incluye pruebas unitarias ni de integración, lo cual es admisible en un contexto académico pero sería insuficiente para una transición a producción.

## 11. Conclusiones y trabajo futuro

### 11.1 Conclusiones

El proyecto Panam, desarrollado por el equipo Data Seekers, demuestra que es viable construir una plataforma de inteligencia de negocio funcional, escalable y estéticamente cuidada utilizando exclusivamente tecnologías de código abierto y servicios de costo cero. La adopción de bases de datos NoSQL, combinada bajo el paradigma de persistencia poliglota, permitió responder a los requerimientos heterogéneos del dominio de retail de calzado mexicano sin las concesiones de desempeño que un modelo relacional puro habría impuesto.

Las contribuciones principales del trabajo son tres. En primer lugar, se desarrolló un *pipeline* reproducible que abarca desde la generación de datos hasta la visualización ejecutiva, con cada etapa documentada y respaldada por código versionado en GitHub. En segundo lugar, se construyó un clasificador de sentimiento propio en español, basado en lexicón curado para el dominio del calzado, que prioriza interpretabilidad sobre precisión absoluta y resulta más adecuado para fines didácticos que las alternativas comerciales. En tercer lugar, se entregaron dos dashboards complementarios —uno en Tableau Public, otro en HTML modular sobre GitHub Pages— que materializan los datos en información accionable bajo principios contemporáneos de visualización analítica.

Desde una perspectiva pedagógica, el proyecto evidencia la importancia de tres prácticas que el equipo internalizó durante el semestre. La primera es la disciplina del control de versiones: cada cambio significativo se introdujo a través de *pull requests* revisados, lo que evitó conflictos mayores y permitió mantener un historial trazable. La segunda es la modularidad del código: la separación clara entre generación, ETL, persistencia, servicios y visualización facilitó la división del trabajo y la incorporación posterior de mejoras. La tercera es el privilegio de la reproducibilidad: la disponibilidad de un `docker-compose.yml` y de scripts de carga independientes garantiza que cualquier evaluador pueda replicar la plataforma localmente.

### 11.2 Trabajo futuro

Las líneas de evolución natural del proyecto se agrupan en cuatro categorías:

1. **Mejora del clasificador de sentimiento.** Sustituir el lexicón por un modelo pre-entrenado

en español (BETO, RoBERTuito) y comparar empíricamente su desempeño frente al enfoque actual. Esta evolución preservaría la interpretabilidad mediante técnicas de explicabilidad como *attention visualization* o LIME.

2. **Despliegue distribuido.** Migrar las bases de datos a configuraciones de *replica set* en MongoDB y *cluster* multi-nodo en Cassandra, con evaluación de tolerancia a fallos y consistencia eventual.
3. **Integración con fuentes reales.** Negociar con la marca Panam el acceso —incluso bajo acuerdos de confidencialidad y agregación— a datos transaccionales auténticos, lo cual permitiría calibrar las distribuciones generadas y validar las decisiones de negocio derivadas del dashboard.
4. **Predicción avanzada de *stockout*.** Sustituir el modelo lineal actual por uno basado en aprendizaje automático (por ejemplo, *gradient boosting* sobre series temporales), incorporando variables exógenas como estacionalidad regional y campañas promocionales.

### 11.3 Reflexión final

La transformación digital del retail no es una promesa de futuro: es una condición de supervivencia del presente. Las marcas que sostengan modelos de gestión basados en intuición, en reportes mensuales y en sistemas relacionales rígidos, verán erosionados sus márgenes frente a competidores capaces de tomar decisiones en minutos sobre la base de datos frescos. La plataforma Panam, en su versión académica de prueba de concepto, busca demostrar que esa transformación es alcanzable con tecnologías accesibles, equipos pequeños y plazos compatibles con un semestre universitario. La barrera, en última instancia, no es tecnológica: es cultural y organizacional.

## Referencias

- Brewer, E. A. (2000). Towards Robust Distributed Systems. *Proceedings of the Annual ACM Symposium on Principles of Distributed Computing*, 7. <https://doi.org/10.1145/343477.343502>
- Carpenter, J., & Hewitt, E. (2020). *Cassandra: The Definitive Guide* (3.<sup>a</sup> ed.). O'Reilly Media.
- Few, S. (2006). *Information Dashboard Design: The Effective Visual Communication of Data*. O'Reilly Media.
- Harrison, G. (2015). *Next Generation Databases: NoSQL, NewSQL, and Big Data*. Apress.
- Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. *Foundations and Trends in Information Retrieval*, 2(1-2), 1-135. <https://doi.org/10.1561/15000000011>
- Sadalage, P. J., & Fowler, M. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2.<sup>a</sup> ed.). Graphics Press.

## A. Anexos

### A.1 Anexo A: Estructura completa del repositorio

```
1 data-seekers/  
2 |-- README.md  
3 |-- docker-compose.yml  
4 |-- requirements.txt  
5 |-- setup.sh  
6 |-- setup.bat  
7 |-- run_api.py  
8 |-- .env.example  
9 |-- .gitignore  
10 |-- api/  
11 |   |-- __init__.py  
12 |   |-- main.py  
13 |   |-- config.py  
14 |   |-- db/  
15 |   |   |-- mongodb.py
```



```
16 | | |-- cassandra.py
17 | | '-- serializers.py
18 | '-- routers/
19 |     |-- kpis.py
20 |     |-- ventas.py
21 |     |-- resenas.py
22 |     |-- inventario.py
23 |     '-- eventos.py
24 |-- dashboards/
25 |     |-- panam_dashboard_v5_FINAL.twbx
26 |     '-- streamlit_dashboard.py
27 |-- data/
28 |     |-- ventas.csv          (83 MB, ignorado en Git)
29 |     |-- resenas.csv        (7.7 MB)
30 |     |-- inventario.csv     (9.6 MB)
31 |     '-- eventos.csv        (41 MB)
32 |-- docs/
33 |     |-- index.html
34 |     |-- README.md
35 |     |-- build_data.py
36 |     |-- css/styles.css
37 |     |-- js/
38 |         |-- utils.js
39 |         |-- state.js
40 |         |-- filters.js
41 |         |-- charts.js
42 |         |-- table.js
43 |         |-- kpis.js
44 |         '-- main.js
45 |     '-- data/panam_data.json
46 |-- notebooks/
47 |     |-- 00_setup_inicial.ipynb
48 |     |-- 01_generacion_datos.ipynb
49 |     |-- 02_etl_transformaciones.ipynb
50 |     |-- 03_mongodb_operaciones_en_vivo.ipynb
51 |     '-- 04_cassandra_consultas_en_vivo.ipynb
```

```
52 |-- scripts/  
53 |   |-- generar_datos_masivos.py  
54 |   |-- carga_datos_mongodb.py  
55 |   '-- carga_datos_cassandra.py  
56 '-- presentacion/  
57   '-- (materiales de la presentacion final)
```

**Listing 1.** Arbol de directorios del repositorio data-seekers

## A.2 Anexo B: Comandos de levantamiento del entorno

```
1 # 1. Clonar el repositorio  
2 git clone https://github.com/CarloKiliano/data-seekers.git  
3 cd data-seekers  
4  
5 # 2. Levantar los servicios NoSQL  
6 docker compose up -d mongodb cassandra  
7  
8 # 3. Verificar que estan corriendo  
9 docker ps  
10  
11 # 4. Activar el entorno virtual de Python e instalar dependencias  
12 python3 -m venv venv  
13 source venv/bin/activate      # En Windows: venv\Scripts\activate  
14 pip install -r requirements.txt  
15  
16 # 5. Generar los datos sinteticos  
17 python scripts/generar_datos_masivos.py --escala full  
18  
19 # 6. Ejecutar el ETL (notebook 02)  
20 jupyter notebook notebooks/02_etl_transformaciones.ipynb  
21  
22 # 7. Cargar a MongoDB y Cassandra  
23 python scripts/carga_datos_mongodb.py  
24 python scripts/carga_datos_cassandra.py  
25  
26 # 8. Levantar el API FastAPI
```

```

27 python run_api.py
28
29 # 9. Abrir el dashboard HTML localmente
30 cd docs && python3 -m http.server 8000
31 # o visitar https://carlokiliano.github.io/data-seekers/

```

Listing 2. Levantamiento completo del entorno

### A.3 Anexo C: Variables del archivo .env.example

```

1 # MongoDB
2 MONGO_URI=mongodb://admin:password@localhost:27017/
3 MONGO_DB=panam_nosql
4
5 # Cassandra
6 CASSANDRA_HOSTS=localhost
7 CASSANDRA_PORT=9042
8 CASSANDRA_KEYSPACE=panam
9
10 # API
11 API_HOST=0.0.0.0
12 API_PORT=8000

```

Listing 3. Variables de entorno del proyecto

### A.4 Anexo D: Información del equipo

Tabla 10. Integrantes del equipo Data Seekers.

| Integrante                        | Rol principal                     | Usuario GitHub |
|-----------------------------------|-----------------------------------|----------------|
| Carlo Kiliano                     | Científico de Datos               | CarloKiliano   |
| Jose Julian Perez Bustamante      | Datos masivos, dashboards, deploy | PBJulian       |
| Leonardo Negrete Mancera          | Data Visualization Analyst        | Leo-25-04      |
| Isaac Pancardo Botello            | Director de Estrategia Comercial  | Botello_Isaac  |
| Aldo Sebastián Altamirano Vázquez | Arquitecto de Datos NoSQL         | V_Altamirano   |

### A.5 Anexo E: Enlaces relevantes

- Repositorio: <https://github.com/CarloKiliano/data-seekers>
- Dashboard HTML en GitHub Pages: <https://carlokiliano.github.io/data-seekers/>
- Documentacion FastAPI: <https://fastapi.tiangolo.com/>
- Documentacion MongoDB: <https://www.mongodb.com/docs/>
- Documentacion Cassandra: <https://cassandra.apache.org/doc/>